

Project 2 : Database Implementation and Application Development

과목 : 데이터베이스시스템

교수 : 정성원 교수님

학과 : 컴퓨터공학과

학번 : 20211562

이름 : 이동협

1. Introduction

Project2는 Project1의 Logical Schema를 넘겨받아 자동차 판매 Database 시스템을 설계하고 구현했다. 데이터베이스는 고객(Customer), 차량(Vehicle), 모델(Model), 브랜드(Brand), 공급업체(Supplier), 생산공장(Manufacturing Plant), 딜러(Dealer), 차량 옵션(Option), 판매 내역(Sales Transaction) 등의 정보를 관리한다.

데이터베이스는 MySQL을 사용하여 구현했고 C++ 프로그램은 MySQL C API를 사용하여 MySQL 데이터베이스와 연동했다.

2. Logical Schema Design

Project 1에서 만든 ER Diagram을 기반으로 Relational Database Schema를 설계했다.

Entity

- Customers
- Dealers
- Brands
- Suppliers
- Manufacturing_Plants
- Models
- Vehicles
- Options
- Sales_Transactions
- Parts

Relationship Table

- Vehicle_Options
- Vehicle_Parts
- Plant_Model

각 Table은 고유한 Primary Key를 가지고 Foreign Key를 이용하여 Table 간의 Relationship을 표현했다.

Customers, Dealers, Brands, Suppliers, Manufacturing_Plants, Models, Vehicles, Options, Sales_Transactions는 ERD의 strong entity를 각각 relation으로 변환했다.

Brands와 Models는 1:N 관계이므로 Models 테이블에 brand_ID를 Foreign Key로 포함했다.

Models와 Vehicles는 1:N 관계이므로 Vehicles 테이블에 model_ID를 Foreign Key로 포함했다.

Manufacturing_Plants와 Vehicles는 1:N 관계이므로 Vehicles 테이블에 plant_ID를 Foreign Key로 포함했다.

Customers, Dealers, Vehicles와 Sales_Transactions의 관계는 판매 거래를 독립 relation으로 두고 customer_ID, dealer_ID, VIN을 Foreign Key로 저장했다.

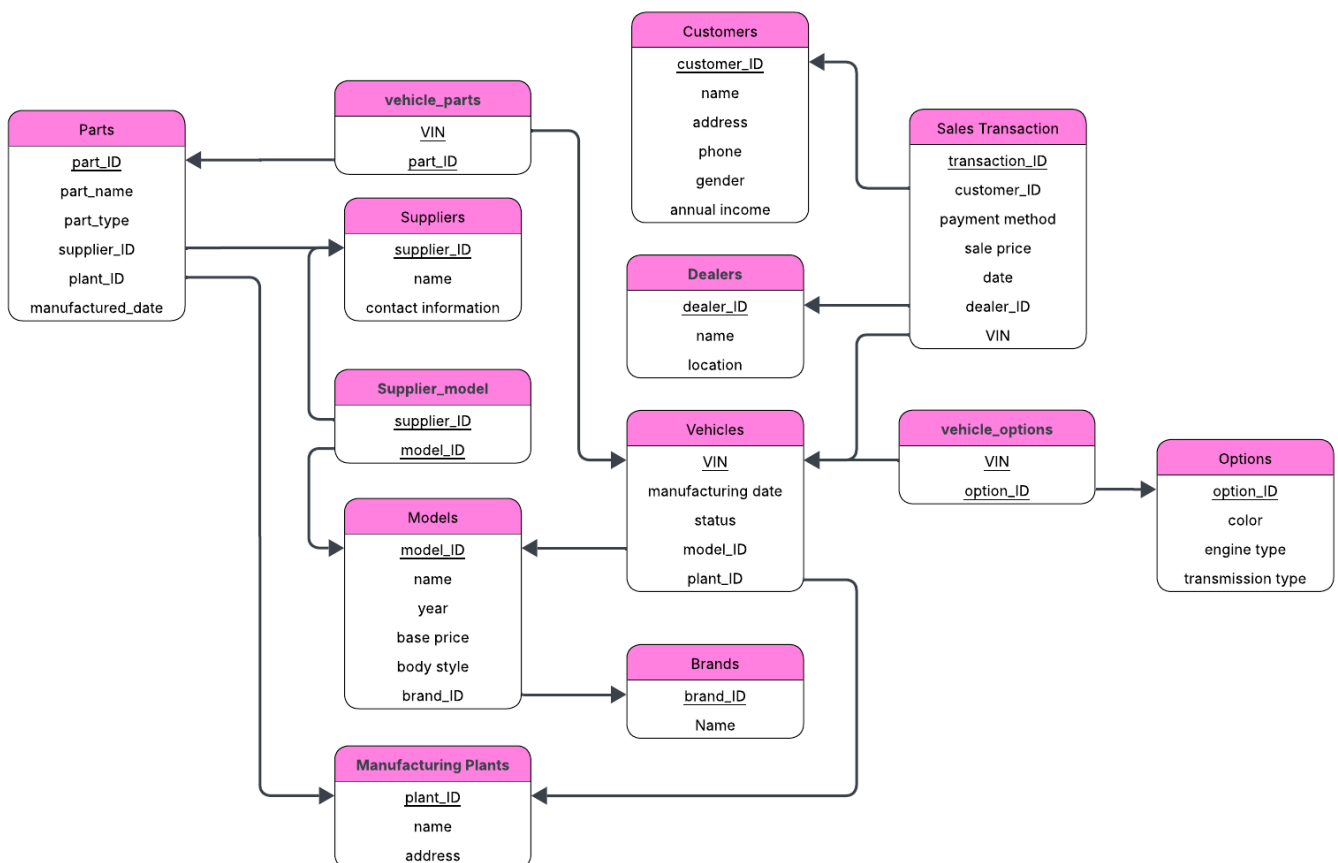
Options와 Vehicles는 M:N 관계이므로 Vehicle_Options relation으로 분해했다.

Suppliers와 Models는 M:N 관계이므로 Supplier_Model relation으로 분해하였다.

Vehicles와 Parts는 M:N 관계이므로 Vehicle_Parts relation으로 분해하였다.

Parts는 supplier_ID와 plant_ID를 Foreign Key로 포함하여 각 부품이 어느 공급업체에 의해 공급되고 어느 생산공장에서 제조되었는지 표현한다.

이러한 분해를 통해 many to many 관계를 관계형 데이터베이스에서 표현하고 중복 저장을 방지했다.



3. BCNF Normalization Analysis

프로젝트의 모든 Table은 BCNF를 만족하도록 설계했다. 기본적으로 모든 Table에 각 항목을 식별하는 ID가 포함되어있어 BCNF를 만족한다.

Customers

Customers(customer_ID, name, address, phone, gender, annual_income)

Functional Dependency:

customer_ID $\rightarrow$ name, address, phone, gender, annual_income

customer_ID는 Customers Table의 PK이므로 각 고객을 유일하게 식별한다. 고객의 이름, 주소, 전화번호, 성별, 연 소득은 모두 특정 고객 ID에 의해 결정된다.

이 relation에서 존재하는 주요 non-trivial FD의 determinant는 customer_ID이며 customer_ID는 candidate key이다. 따라서 모든 non-trivial FD의 determinant가 superkey이므로 Customers는 BCNF를 만족한다.

Dealers

Dealers(dealer_ID, name, location)

Functional Dependency:

dealer_ID $\rightarrow$ name, location

dealer_ID는 Dealers Table의 PK이며 각 딜러를 유일하게 식별한다. 딜러의 이름과 위치는 dealer_ID에 의해 결정된다.

dealer_ID가 candidate key이므로 FD dealer_ID $\rightarrow$ name, location의 determinant는 superkey이다. 따라서 Dealers Table은 BCNF를 만족한다.

Brands

Brands(brand_ID, name)

Functional Dependency:

brand_ID $\rightarrow$ name

brand_ID는 Brands Table의 PK이다. 각 브랜드 ID는 하나의 브랜드 이름을 결정한다. 또한 실제 구현에서는 브랜드 이름의 중복을 방지하기 위해 name에 UNIQUE 제약 조건을 두었다.

따라서 brand_ID는 candidate key이며, brand_ID $\rightarrow$ name의 determinant가 superkey이므로 Brands는

BCNF를 만족한다. 또한 name이 UNIQUE이므로 $\text{name} \rightarrow \text{brand_ID}$ 도 성립할 수 있으며, 이 경우에도 name은 candidate key가 되어 BCNF 조건을 위반하지 않는다.

Models

Models(model_ID, brand_ID, name, year, base_price, body_style)

Functional Dependency:

$\text{model_ID} \rightarrow \text{brand_ID}, \text{name}, \text{year}, \text{base_price}, \text{body_style}$

model_ID는 Models Table의 PK며 각 차량 모델을 유일하게 식별한다. 특정 모델 ID가 주어지면 해당 모델이 속한 브랜드, 모델명, 연식, 기본 가격, 차체 스타일이 결정된다.

이 relation에서 주요 FD의 determinant는 model_ID이다. model_ID는 candidate key이므로 superkey이다. 따라서 $\text{model_ID} \rightarrow \text{brand_ID}, \text{name}, \text{year}, \text{base_price}, \text{body_style}$ 은 BCNF 조건을 만족한다.

브랜드와 모델의 1:N 관계는 Models Table에 brand_ID를 Foreign Key로 포함하여 표현하였다. brand_ID는 여러 모델에서 반복될 수 있으므로 brand_ID 단독으로는 name, year, base_price, body_style을 결정하지 않는다. 따라서 brand_ID로 인한 BCNF 위반은 발생하지 않는다.

Vehicles

Vehicles(VIN, model_ID, plant_ID, manufacturing_date, status)

Functional Dependency:

$\text{VIN} \rightarrow \text{model_ID}, \text{plant_ID}, \text{manufacturing_date}, \text{status}$

VIN은 차량의 고유 식별 번호이며 Vehicles Table의 PK이다. 하나의 VIN은 정확히 하나의 차량을 나타내므로 해당 차량의 모델, 생산 공장, 제조일, 상태를 결정한다.

FD $\text{VIN} \rightarrow \text{model_ID}, \text{plant_ID}, \text{manufacturing_date}, \text{status}$ 에서 determinant인 VIN은 candidate key이다. 따라서 determinant가 superkey이므로 Vehicles Table은 BCNF를 만족한다.

model_ID와 plant_ID는 각각 Models와 Manufacturing_Plants를 참조하는 Foreign Key이다. 하지만 하나의 모델은 여러 차량에 사용될 수 있고 하나의 공장도 여러 차량을 생산할 수 있으므로 model_ID나 plant_ID만으로 특정 차량의 모든 속성을 결정할 수 없다. 따라서 이 FK들은 BCNF 위반을 만들지 않는다.

Sales_Transactions

Sales_Transactions(transaction_ID, customer_ID, dealer_ID, VIN, date, payment_method, sale_price)

Functional Dependency:

transaction_ID → customer_ID, dealer_ID, VIN, date, payment_method, sale_price

VIN → transaction_ID, customer_ID, dealer_ID, date, payment_method, sale_price

transaction_ID는 Sales_Transactions Table의 PK며 각 판매 거래를 유일하게 식별한다. 따라서 특정 거래 ID가 주어지면 구매 고객, 판매 딜러, 판매 차량, 판매일, 결제 방식, 판매 가격이 결정된다.

또한 구현에서 VIN에는 UNIQUE 제약 조건을 설정하였다. 이는 하나의 차량이 최대 한 번만 판매될 수 있다는 규칙을 반영하기 위한 것이다. 따라서 VIN도 Sales_Transactions Table에서 candidate key로 볼 수 있다.

이 relation에서 주요 non-trivial FD의 determinant는 transaction_ID 또는 VIN이다. 두 determinant 모두 candidate key이므로 superkey이다. 따라서 Sales_Transactions Table은 BCNF를 만족한다.

customer_ID와 dealer_ID는 Foreign Key이지만 한 고객이나 한 딜러가 여러 판매 거래에 참여할 수 있으므로 각각 단독으로는 거래 전체를 결정하지 못한다. 따라서 이들 속성으로 인한 BCNF 위반은 없다.

Suppliers

Suppliers(supplier_ID, name, contact_information)

Functional Dependency:

supplier_ID → name, contact_information

supplier_ID는 Suppliers Table Primary Key이며 각 공급업체를 유일하게 식별한다. 공급업체 이름과 연락처 정보는 supplier_ID에 의해 결정된다.

FD supplier_ID → name, contact_information에서 determinant인 supplier_ID는 candidate key이다. 따라서 Suppliers Table은 BCNF를 만족한다.

Manufacturing_Plants

Manufacturing_Plants(plant_ID, name, address)

Functional Dependency:

plant_ID → name, address

plant_ID는 Manufacturing_Plants Table의 PK며 각 생산 공장을 유일하게 식별한다. 특정 공장 ID가 주어지면 공장명과 주소가 결정된다.

FD plant_ID → name, address의 determinant인 plant_ID는 candidate key이므로 superkey이다. 따라서 Manufacturing_Plants Table은 BCNF를 만족한다.

Options

Options(option_ID, color, engine_type, transmission_type)

Functional Dependency:

option_ID $\rightarrow$ color, engine_type, transmission_type

option_ID는 Options Table의 Primary Key이며, 각 차량 옵션 조합을 유일하게 식별한다. 특정 옵션 ID는 색상, 엔진 타입, 변속기 타입을 결정한다.

FD option_ID $\rightarrow$ color, engine_type, transmission_type에서 determinant인 option_ID는 candidate key이다. 따라서 Options Table은 BCNF를 만족한다.

색상, 엔진 타입, 변속기 타입은 여러 옵션 조합에서 반복될 수 있으므로 각각 단독으로는 option_ID를 결정하지 않는다. 따라서 추가적인 BCNF 위반은 발생하지 않는다.

Parts

Parts(part_ID, part_name, part_type, supplier_ID, plant_ID, manufactured_date)

Functional Dependency:

part_ID $\rightarrow$ part_name, part_type, supplier_ID, plant_ID, manufactured_date

part_ID는 Parts Table의 Primary Key이며 각 부품을 유일하게 식별한다. 특정 part_ID가 주어지면 부품명, 부품 종류, 공급업체, 제조 공장, 제조일이 결정된다. supplier_ID와 plant_ID는 각각 Suppliers와 Manufacturing_Plants를 참조하는 Foreign Key이지만 하나의 공급업체나 공장은 여러 부품을 공급 또는 생산할 수 있으므로 단독으로 part_ID를 결정하지 않는다. 따라서 Parts Table은 BCNF를 만족한다.

Relationship Tables

Vehicle_Options

Vehicle_Options(VIN, option_ID)

Functional Dependency:

(VIN, option_ID) $\rightarrow$ all attributes

Vehicle_Options는 Vehicles와 Options 사이의 M:N 관계를 표현하기 위한 relation이다. 복합 Primary Key (VIN, option_ID)를 사용하여 하나의 차량과 하나의 옵션 조합이 중복 저장되지 않도록 하였다.

이 Table에는 복합 키 이외의 일반 속성이 존재하지 않는다. 따라서 non-trivial FD가 없거나 determinant가 전체 key인 경우뿐이므로 BCNF를 만족한다.

Supplier_Model

Supplier_Model(supplier_ID, model_ID)

Functional Dependency:

(supplier_ID, model_ID) → all attributes

Supplier_Model은 Suppliers와 Models 사이의 M:N 관계를 표현한다. 하나의 공급업체가 여러 모델에 부품을 공급할 수 있고, 하나의 모델도 여러 공급업체로부터 부품을 공급받을 수 있으므로 별도의 관계 Table로 분해하였다.

복합 Primary Key (supplier_ID, model_ID)가 각 tuple을 유일하게 식별하며, 추가적인 일반 속성이 없으므로 BCNF를 만족한다.

Vehicle_Parts

Vehicle_Parts(VIN, part_ID)

Functional Dependency:

(VIN, part_ID) → all attributes

Vehicle_Parts는 Vehicles와 Parts 사이의 M:N 관계를 표현하기 위한 relation이다. 하나의 차량은 여러 부품을 포함할 수 있고 부품 데이터는 차량과 독립적으로 관리되므로 별도 관계 Table로 분해했다. 복합 Primary Key (VIN, part_ID)가 각 tuple을 유일하게 식별하며 추가적인 일반 속성이 없으므로 BCNF를 만족한다.

전체 relation을 검토한 결과 모든 non-trivial functional dependency의 determinant는 Primary Key, Candidate Key, 또는 복합 Primary Key이다. 따라서 본 프로젝트의 최종 relational schema는 BCNF를 만족한다.

4. Physical Database Implementation

데이터베이스는 MySQL 8.0 환경에서 구현했다.

총 13개의 Table을 생성하였으며 주요 data type은 다음과 같다.

- INT : 식별자
- VARCHAR : 문자열 데이터
- DATE : 날짜 데이터
- DECIMAL : 금액 및 소득 정보
- ENUM : 상태 정보

customer_ID, dealer_ID, brand_ID, supplier_ID, plant_ID, model_ID, option_ID, transaction_ID는 각 tuple을 식별하기 위한 surrogate key로 INT를 사용했다.

VIN은 실제 차량 식별 번호를 표현하기 위해 VARCHAR(17)을 사용하고 Vehicles 테이블의 Primary Key로 지정했다.

sale_price, base_price, annual_income은 금액 또는 소득 데이터이므로 DECIMAL(12,2)을 사용하여 부동소수점 오차를 방지했다.

date와 manufacturing_date는 날짜 기반 분석과 DATEDIFF 계산이 필요하므로 DATE 타입을 사용했다.

gender와 vehicle status는 허용 가능한 값이 제한되어 있으므로 ENUM 타입을 사용했다.

Brands.name은 UNIQUE 제약 조건을 두어 동일한 브랜드명이 중복 저장되는 것을 방지했다.

Sales_Transactions.VIN은 UNIQUE로 설정하여 하나의 차량이 여러 번 판매되는 것을 방지했다.

Vehicle_Options, Vehicle_Parts, Supplier_Model은 복합 Primary Key를 사용하여 동일한 관계가 중복 저장되지 않도록 했다.

part_ID는 부품 식별자로 INT AUTO_INCREMENT를 사용하였다.

part_name과 part_type은 문자열 정보이므로 VARCHAR를 사용하였다.

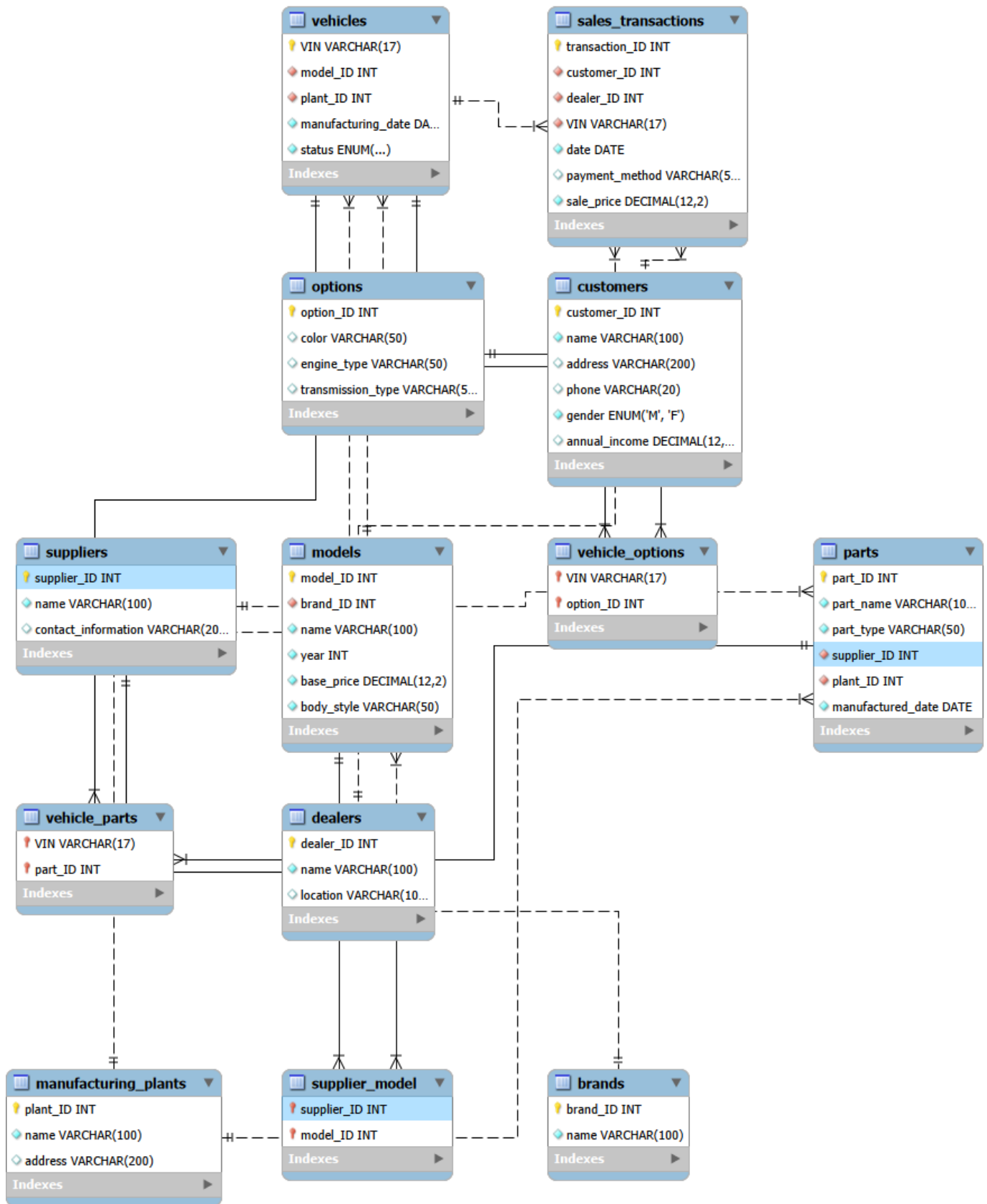
manufactured_date는 결함 부품 추적에서 날짜 범위 조건에 사용되므로 DATE 타입을 사용하였다.

Parts.supplier_ID와 Parts.plant_ID는 각각 Suppliers와 Manufacturing_Plants를 참조하는 Foreign Key이다.

모든 Foreign Key는 참조 무결성을 보장하며 존재하지 않는 customer_ID, dealer_ID, VIN, model_ID 등을 참조하는 데이터 삽입은 실패한다.

Vehicle_Parts는 존재하지 않는 VIN이나 part_ID를 참조할 수 없도록 Foreign Key 제약 조건을 가진다.

Parts는 존재하지 않는 supplier_ID나 plant_ID를 참조할 수 없도록 제약 조건을 둔다.



5. Application Development

C++과 MySQL C API를 이용하여 데이터베이스와 연동되는 프로그램을 구현하였다.

프로그램은 다음과 같은 기능으로 구성된다.

Database Connection

MySQL C API의 `mysql_init()`, `mysql_real_connect()`, `mysql_query()`, `mysql_store_result()` 함수를 사용하여 데이터베이스에 연결하고 SQL 질의를 실행한다.

데이터베이스 접속 정보는 `MYSQL_HOST`, `MYSQL_USER`, `MYSQL_PASSWORD`, `MYSQL_DATABASE`, `MYSQL_PORT` 환경변수를 통해 설정할 수 있으며 환경변수가 없는 경우 기본값을 사용한다.

`mysql_init()`으로 MySQL connection handler를 초기화하고 `mysql_real_connect()`를 통해 MySQL 서버에 연결한다. 접속 실패 시 `mysql_error()`를 이용해 오류 메시지를 출력한다.

Query Execution

사용자가 메뉴를 선택하면 해당 SQL 질의를 실행하고 결과를 출력한다.

Menu Interface

사용자가 원하는 분석 기능을 선택할 수 있도록 메뉴 기반 인터페이스를 제공한다.

프로그램 구조는 다음과 같다.

- `database.h`
- `main.cpp`

`database.h`는 데이터베이스 관련 함수 선언을 포함하고 있으며 `main.cpp`는 메뉴 처리 및 SQL 실행 기능을 구현한다.

6. Query Implementation

Query 1. Sales Trends Analysis

최근 3년 동안의 판매 데이터를 브랜드별로 분석한다. 판매 정보는 Sales_Transactions Table에서 가져오고 판매 차량의 브랜드를 확인하기 위해 Vehicles, Models, Brands Table을 순서대로 JOIN한다. 또한 구매자의 성별과 소득 구간을 분석하기 위해 Customers Table을 JOIN한다.

하나의 결과만 출력하는 것이 아니라 다음 6가지 기준으로 나누어 판매 추이를 확인한다.

- Year / Gender
- Year / Income Range
- Month / Gender
- Month / Income Range
- Week / Gender
- Week / Income Range

YEAR, MONTH, WEEK 함수를 사용하여 판매일을 연도, 월, 주 단위로 분류한다. 성별 분석에서는 Customers.gender를 기준으로 그룹화하고 소득 구간 분석에서는 annual_income 값을 기준으로 Low, Middle, High 구간을 나누어 그룹화한다.

출력 정보는 브랜드명, 기간 정보, 성별 또는 소득 구간, 판매 대수, 총 판매 금액이다.

Query 2. Defective Part Tracking

특정 공급업체가 제조한 특정 부품 종류가 결함으로 확인되었을 때 해당 부품이 장착된 차량과 판매 고객을 추적한다. 먼저 Suppliers에서 공급업체 이름이 Getrag인 tuple을 찾고 Parts Table과 JOIN하여 Getrag가 공급한 부품 중 part_type이 Transmission인 부품을 선택한다. 또한 Parts.manufactured_date를 이용하여 결함 부품 제조일이 지정된 두 날짜 사이에 포함되는지 필터링한다.

이후 Vehicle_Parts를 통해 해당 부품이 장착된 차량의 VIN을 찾고 Vehicles, Sales_Transactions, Customers와 JOIN하여 해당 차량이 판매된 고객의 이름, 전화번호, 주소를 확인한다. Manufacturing_Plants와도 JOIN하여 부품이 생산된 공장명을 함께 출력한다.

또한 결함 추적 조건으로 부품의 제조일이 지정된 두 날짜 사이에 포함되는지 확인한다.

출력 정보

- VIN
- Customer Name
- Phone
- Address

- Supplier Name
- Plant Name
- Part Name
- Part Type
- Part Manufacturing Date
- Sale Date

Query 3. Top 2 Brands by Revenue

최근 1년 동안의 판매 데이터를 기준으로 총 매출액이 가장 높은 브랜드 2개를 조회한다. 판매 가격은 Sales_Transactions.sale_price에서 가져오며, 해당 판매 차량의 브랜드를 찾기 위해 Vehicles, Models, Brands Table을 JOIN한다.

SUM(st.sale_price)를 사용하여 브랜드별 총 매출액을 계산하고 GROUP BY b.name으로 브랜드 단위 집계를 수행한다. 이후 총 매출액을 기준으로 내림차순 정렬하고 LIMIT 2를 사용하여 상위 2개 브랜드만 출력한다.

출력 정보

- Brand Name
- Total Revenue

Query 4. Top 2 Brands by Unit Sales

최근 1년 동안의 판매 데이터를 기준으로 판매 대수가 가장 많은 브랜드 2개를 조회한다. 판매 거래는 Sales_Transactions Table에서 가져오고 차량의 브랜드를 확인하기 위해 Vehicles, Models, Brands Table을 JOIN한다.

COUNT(*)를 사용하여 브랜드별 판매 대수를 계산하고 GROUP BY b.name으로 브랜드별 집계를 수행한다. 이후 판매 대수를 기준으로 내림차순 정렬한 뒤 LIMIT 2를 사용하여 판매량 상위 2개 브랜드만 출력한다.

출력 정보

- Brand Name
- Units Sold

Query 5. Seasonal Sales Patterns

특정 차체 스타일, 예를 들어 Convertible 차량이 가장 많이 판매된 월을 조회한다. 판매 내역은

Sales_Transactions Table에서 가져오고 판매된 차량의 차체 스타일을 확인하기 위해 Vehicles와 Models Table을 조인한다.

Models.body_style = 'Convertible' 조건을 사용하여 Convertible 차량만 선택한다. 이후 MONTH(st.date)를 사용하여 판매 월을 추출하고, 월별 판매 대수를 COUNT(*)로 계산한다. 월별 판매량을 내림차순 정렬하고 LIMIT 1을 사용하여 가장 많이 판매된 월을 출력한다.

출력 정보

- Sale Month
- Units Sold

Query 6. Dealer Inventory Efficiency

딜러별 평균 재고 보유 기간을 계산하여 어떤 딜러가 차량을 가장 오래 보유했는지 분석한다. 본 설계에는 별도의 재고 입고일이 존재하지 않기 때문에 차량의 제조일을 재고 시작 시점으로 보고 판매일까지의 차이를 재고 보유 기간으로 계산하였다.

먼저 Dealers Table과 Sales_Transactions Table을 dealer_ID 기준으로 JOIN하여 각 판매 거래를 담당한 딜러를 찾는다. 이후 Sales_Transactions와 Vehicles를 VIN 기준으로 JOIN하여 판매된 차량의 제조일을 가져온다.

DATEDIFF(st.date, v.manufacturing_date)를 사용하여 각 차량이 제조된 날부터 판매된 날까지의 일수를 계산한다. 그리고 AVG() 함수를 사용하여 딜러별 평균 재고 보유 기간을 계산한다. 마지막으로 평균 보유 기간이 긴 순서대로 정렬하여 차량을 가장 오래 보유한 딜러를 확인한다.

출력 정보

- Dealer Name
- Average Inventory Days

Query 7. Supplier Coverage

가장 많은 차량 모델에 부품을 공급하는 공급업체를 조회한다. 공급업체 정보는 Suppliers 테이블에서 가져오고 공급업체와 모델 사이의 관계는 Supplier_Model 관계 Table을 통해 확인한다.

COUNT(DISTINCT sm.model_ID)를 사용하여 각 공급업체가 공급하는 서로 다른 모델 수를 계산한다. 이후 공급업체별로 그룹화하고 모델 수를 기준으로 내림차순 정렬한 뒤 LIMIT 1을 사용하여 가장 많은 모델을 담당하는 공급업체를 출력한다.

출력 정보

- Supplier Name
- Model Count

7. 테스트 및 검증

모든 SQL 질의를 MySQL Workbench에서 먼저 검증한 후 C++ 프로그램에서 동일하게 실행했다.

Test Case 1: schema.sql 실행

Expected Result: 모든 테이블과 Foreign Key 제약 조건이 정상적으로 생성된다.

Actual Result: 성공

Test Case 2: sample_data.sql 실행

Expected Result: 샘플 데이터가 모든 테이블에 정상적으로 삽입된다.

Actual Result: 성공

Test Case 3: 메뉴 1~7 실행

Expected Result: 각 sample query의 결과가 콘솔에 출력된다.

Actual Result: 성공

Test Case 4: 잘못된 메뉴 번호 입력

Expected Result: Invalid number selected 메시지를 출력하고 메뉴를 다시 표시한다.

Actual Result: 성공

Business Rule Validation:

존재하지 않는 customer_ID, dealer_ID, VIN을 Sales_Transactions에 삽입하면 Foreign Key 제약 조건으로 실패한다. 동일한 VIN을 Sales_Transactions에 두 번 삽입하면 UNIQUE 제약 조건으로 실패한다. Vehicle_Options, Vehicle_Parts, Supplier_Model은 복합 Primary Key를 사용하므로 동일 관계의 중복 삽입을 방지한다.

또한, 존재하지 않는 supplier_ID 또는 plant_ID를 가진 Parts tuple은 Foreign Key 제약 조건으로 삽입되지 않는다. 또한 존재하지 않는 VIN 또는 part_ID를 가진 Vehicle_Parts tuple도 삽입되지 않는다.

===== Sample Query Menu =====

1. Sales Trends
2. Defective Part Tracking
3. Top 2 Brands by Revenue
4. Top 2 Brands by Unit Sales
5. Seasonal Sales Patterns
6. Dealer Inventory Efficiency
7. Supplier Coverage
0. Exit

Select: 1

===== Year / Gender =====

brand	sale_year	gender	units_sold	total_sales
-----+-----+-----+-----+-----				
Ferrari	2023	F	1	69000.00
Ferrari	2024	M	1	48000.00
Porsche	2024	M	1	46000.00
Porsche	2024	F	1	36000.00
Aston Martin	2025	M	1	252000.00
BMW	2025	M	1	89000.00
BMW	2025	F	1	101000.00
Bugatti	2025	M	1	3150000.00
Chevrolet	2025	M	1	72000.00
Dodge	2025	F	1	78000.00
Ferrari	2025	M	2	453000.00
Ferrari	2025	F	1	55000.00
Koenigsegg	2025	F	1	3100000.00
Lamborghini	2025	F	1	255000.00
Lotus	2025	M	1	103000.00
Maserati	2025	F	1	247000.00
McLaren	2025	F	1	328000.00
Pagani	2025	F	1	3500000.00
Porsche	2025	M	1	128000.00
Alfa Romeo	2026	F	1	86000.00
Audi	2026	M	1	165000.00
Bentley	2026	F	1	252000.00
Ferrari	2026	M	1	68000.00
Ford	2026	M	1	51000.00
Genesis	2026	F	1	56000.00
Jaguar	2026	M	1	119000.00
Mercedes-AMG	2026	F	1	142000.00
Porsche	2026	F	1	42000.00

===== Year / Income Range =====				
brand	sale_year	income_range	units_sold	total_sales
-----+-----+-----+-----+-----				
Ferrari	2023	Middle	1	69000.00
Ferrari	2024	Middle	1	48000.00
Porsche	2024	High	2	82000.00
Aston Martin	2025	High	1	252000.00
BMW	2025	Middle	2	190000.00
Bugatti	2025	Middle	1	3150000.00
Chevrolet	2025	High	1	72000.00
Dodge	2025	High	1	78000.00
Ferrari	2025	High	2	163000.00
Ferrari	2025	Low	1	345000.00
Koenigsegg	2025	High	1	3100000.00
Lamborghini	2025	Middle	1	255000.00
Lotus	2025	Middle	1	103000.00
Maserati	2025	High	1	247000.00
McLaren	2025	High	1	328000.00
Pagani	2025	Middle	1	3500000.00
Porsche	2025	Middle	1	128000.00
Alfa Romeo	2026	High	1	86000.00
Audi	2026	High	1	165000.00
Bentley	2026	Middle	1	252000.00
Ferrari	2026	Middle	1	68000.00
Ford	2026	Middle	1	51000.00
Genesis	2026	Middle	1	56000.00
Jaguar	2026	High	1	119000.00
Mercedes-AMG	2026	High	1	142000.00
Porsche	2026	High	1	42000.00

===== Month / Gender =====					
brand	sale_year	sale_month	gender	units_sold	total_sales
-----+-----+-----+-----+-----+-----					
Ferrari	2023	8	F	1	69000.00
Ferrari	2024	2	M	1	48000.00
Porsche	2024	5	F	1	36000.00
Porsche	2024	10	M	1	46000.00
BMW	2025	3	F	1	101000.00
Ferrari	2025	6	M	1	345000.00
Lamborghini	2025	6	F	1	255000.00
Ferrari	2025	7	M	1	108000.00
McLaren	2025	7	F	1	328000.00
Porsche	2025	7	M	1	128000.00
Aston Martin	2025	8	M	1	252000.00
Maserati	2025	8	F	1	247000.00
Bugatti	2025	9	M	1	3150000.00
Koenigsegg	2025	9	F	1	3100000.00
Lotus	2025	10	M	1	103000.00
Pagani	2025	10	F	1	3500000.00
Chevrolet	2025	11	M	1	72000.00
BMW	2025	12	M	1	89000.00
Dodge	2025	12	F	1	78000.00
Ferrari	2025	12	F	1	55000.00
Mercedes-AMG	2026	1	F	1	142000.00
Audi	2026	2	M	1	165000.00
Bentley	2026	2	F	1	252000.00
Ferrari	2026	2	M	1	68000.00
Ford	2026	3	M	1	51000.00
Alfa Romeo	2026	4	F	1	86000.00
Jaguar	2026	4	M	1	119000.00
Porsche	2026	4	F	1	42000.00
Genesis	2026	5	F	1	56000.00

===== Month / Income Range =====					
brand	sale_year	sale_month	income_range	units_sold	total_sales
Ferrari	2023	8	Middle	1	69000.00
Ferrari	2024	2	Middle	1	48000.00
Porsche	2024	5	High	1	36000.00
Porsche	2024	10	High	1	46000.00
BMW	2025	3	Middle	1	101000.00
Ferrari	2025	6	Low	1	345000.00
Lamborghini	2025	6	Middle	1	255000.00
Ferrari	2025	7	High	1	108000.00
McLaren	2025	7	High	1	328000.00
Porsche	2025	7	Middle	1	128000.00
Aston Martin	2025	8	High	1	252000.00
Maserati	2025	8	High	1	247000.00
Bugatti	2025	9	Middle	1	3150000.00
Koenigsegg	2025	9	High	1	3100000.00
Lotus	2025	10	Middle	1	103000.00
Pagani	2025	10	Middle	1	3500000.00
Chevrolet	2025	11	High	1	72000.00
BMW	2025	12	Middle	1	89000.00
Dodge	2025	12	High	1	78000.00
Ferrari	2025	12	High	1	55000.00
Mercedes-AMG	2026	1	High	1	142000.00
Audi	2026	2	High	1	165000.00
Bentley	2026	2	Middle	1	252000.00
Ferrari	2026	2	Middle	1	68000.00
Ford	2026	3	Middle	1	51000.00
Alfa Romeo	2026	4	High	1	86000.00
Jaguar	2026	4	High	1	119000.00
Porsche	2026	4	High	1	42000.00
Genesis	2026	5	Middle	1	56000.00

===== Week / Gender =====					
brand	sale_year	sale_week	gender	units_sold	total_sales
Ferrari	2023	33	F	1	69000.00
Ferrari	2024	6	M	1	48000.00
Porsche	2024	21	F	1	36000.00
Porsche	2024	42	M	1	46000.00
BMW	2025	12	F	1	101000.00
Ferrari	2025	23	M	1	345000.00
Lamborghini	2025	25	F	1	255000.00
Porsche	2025	27	M	1	128000.00
McLaren	2025	29	F	1	328000.00
Ferrari	2025	30	M	1	108000.00
Aston Martin	2025	31	M	1	252000.00
Maserati	2025	34	F	1	247000.00
Bugatti	2025	37	M	1	3150000.00
Koenigsegg	2025	39	F	1	3100000.00
Pagani	2025	40	F	1	3500000.00
Lotus	2025	43	M	1	103000.00
Chevrolet	2025	45	M	1	72000.00
Dodge	2025	48	F	1	78000.00
Ferrari	2025	50	F	1	55000.00
BMW	2025	51	M	1	89000.00
Mercedes-AMG	2026	3	F	1	142000.00
Audi	2026	5	M	1	165000.00
Bentley	2026	8	F	1	252000.00
Ferrari	2026	8	M	1	68000.00
Ford	2026	10	M	1	51000.00
Alfa Romeo	2026	13	F	1	86000.00
Jaguar	2026	16	M	1	119000.00
Porsche	2026	17	F	1	42000.00
Genesis	2026	19	F	1	56000.00

===== Week / Income Range =====					
brand	sale_year	sale_week	income_range	units_sold	total_sales
Ferrari	2023	33	Middle	1	69000.00
Ferrari	2024	6	Middle	1	48000.00
Porsche	2024	21	High	1	36000.00
Porsche	2024	42	High	1	46000.00
BMW	2025	12	Middle	1	101000.00
Ferrari	2025	23	Low	1	345000.00
Lamborghini	2025	25	Middle	1	255000.00
Porsche	2025	27	Middle	1	128000.00
McLaren	2025	29	High	1	328000.00
Ferrari	2025	30	High	1	108000.00
Aston Martin	2025	31	High	1	252000.00
Maserati	2025	34	High	1	247000.00
Bugatti	2025	37	Middle	1	3150000.00
Koenigsegg	2025	39	High	1	3100000.00
Pagani	2025	40	Middle	1	3500000.00
Lotus	2025	43	Middle	1	103000.00
Chevrolet	2025	45	High	1	72000.00
Dodge	2025	48	High	1	78000.00
Ferrari	2025	50	High	1	55000.00
BMW	2025	51	Middle	1	89000.00
Mercedes-AMG	2026	3	High	1	142000.00
Audi	2026	5	High	1	165000.00
Bentley	2026	8	Middle	1	252000.00
Ferrari	2026	8	Middle	1	68000.00
Ford	2026	10	Middle	1	51000.00
Alfa Romeo	2026	13	High	1	86000.00
Jaguar	2026	16	High	1	119000.00
Porsche	2026	17	High	1	42000.00
Genesis	2026	19	Middle	1	56000.00

===== Sample Query Menu =====

- 1. Sales Trends
- 2. Defective Part Tracking
- 3. Top 2 Brands by Revenue
- 4. Top 2 Brands by Unit Sales
- 5. Seasonal Sales Patterns
- 6. Dealer Inventory Efficiency
- 7. Supplier Coverage
- 0. Exit

Select: 2

VIN	customer_name	phone	address	supplier_name	plant_name	part_name	part_type	part_manufactured_date	sale_date
VIN20250000000001	Minsoo Kim	010-1001-1001	Seoul	Getrag	Seoul Plant	GT Automatic Transmission A1	Transmission	2025-03-20	2025-06-12
VIN20250000000002	Jiyoung Lee	010-1002-1002	Busan	Getrag	Busan Plant	GT Automatic Transmission A2	Transmission	2025-04-02	2025-06-28
VIN20250000000003	Junho Park	010-1003-1003	Incheon	Getrag	Incheon Plant	GT Automatic Transmission A3	Transmission	2025-04-18	2025-07-09
VIN20250000000004	Mina Choi	010-1004-1004	Daegu	Getrag	Daegu Plant	GT Automatic Transmission A4	Transmission	2025-05-01	2025-07-22

===== Sample Query Menu =====

- 1. Sales Trends
- 2. Defective Part Tracking
- 3. Top 2 Brands by Revenue
- 4. Top 2 Brands by Unit Sales
- 5. Seasonal Sales Patterns
- 6. Dealer Inventory Efficiency
- 7. Supplier Coverage
- 0. Exit

Select: 3

brand | total_revenue

-----+-----

Pagani | 3500000.00

Bugatti | 3150000.00

===== Sample Query Menu =====

- 1. Sales Trends
- 2. Defective Part Tracking
- 3. Top 2 Brands by Revenue
- 4. Top 2 Brands by Unit Sales
- 5. Seasonal Sales Patterns
- 6. Dealer Inventory Efficiency
- 7. Supplier Coverage
- 0. Exit

Select: 4

brand | units_sold

-----+-----

Ferrari | 4

Porsche | 2

===== Sample Query Menu =====

1. Sales Trends
2. Defective Part Tracking
3. Top 2 Brands by Revenue
4. Top 2 Brands by Unit Sales
5. Seasonal Sales Patterns
6. Dealer Inventory Efficiency
7. Supplier Coverage
0. Exit

Select: 5

sale_month | units_sold

-----+-----
4 | 2

===== Sample Query Menu =====

1. Sales Trends
2. Defective Part Tracking
3. Top 2 Brands by Revenue
4. Top 2 Brands by Unit Sales
5. Seasonal Sales Patterns
6. Dealer Inventory Efficiency
7. Supplier Coverage
0. Exit

Select: 6

dealer | avg_inventory_days

-----+-----
Changwon Motor Park | 63.0000
Cheongju Auto Plaza | 51.0000
Yuseong Premium Auto | 49.0000
Jeonju Auto Gallery | 47.0000
Suwon Speed Motors | 45.0000
Ilsan Premium Auto | 45.0000
Wonju Auto Square | 45.0000
Ulsan Auto Prestige | 44.0000
Gangnam Hypercar Center | 43.0000
Bundang Motor Studio | 43.0000
Sangmu Sports Garage | 43.0000
Mapo Performance Hub | 42.0000
Dongseongro Speed Shop | 41.0000
Haeundae Auto Lounge | 40.0000
Busan Premium Motors | 39.0000
Cheonan Prestige Cars | 39.0000
Mokpo Grand Motors | 39.0000
Gwangju Performance Cars | 38.0000
Daegu Sports Auto | 37.0000
Daejeon Luxury Motors | 37.0000
Jeju Grand Motors | 37.0000
Pohang Auto House | 37.0000
Incheon Exotic Cars | 34.0000
Seoul Supercar Gallery | 33.0000
Pangyo Mobility House | 32.0000
Nambu Drive Center | 31.0000
Anyang Sports Motors | 30.0000
Songdo Motor Arena | 21.0000
Gimhae Auto World | 20.0000
Sejong Luxury Auto | 11.0000

===== Sample Query Menu =====

1. Sales Trends
2. Defective Part Tracking
3. Top 2 Brands by Revenue
4. Top 2 Brands by Unit Sales
5. Seasonal Sales Patterns
6. Dealer Inventory Efficiency
7. Supplier Coverage
0. Exit

Select: 7

supplier | model_count

-----+-----
Getrag | 4